

Reinforcement Learning-based Fault-Tolerant Control for Quadrotor with Online Transformer Adaptation

Dohyun Kim¹ , Jayden Dongwoo Lee² , Hyochoong Bang²  and Jungho Bae¹ 

Abstract—Multirotors play a significant role in diverse field robotics applications but remain highly susceptible to actuator failures, leading to rapid instability and compromised mission reliability. While various fault-tolerant control (FTC) strategies using reinforcement learning (RL) have been widely explored, most previous approaches require prior knowledge of the multirotor model or struggle to adapt to new configurations. To address these limitations, we propose a novel hybrid RL-based FTC framework integrated with a transformer-based online adaptation module. Our framework leverages a transformer architecture to infer latent representations in real time, enabling adaptation to previously unseen system models without retraining. We evaluate our method in a PyBullet simulation under loss-of-effectiveness actuator faults, achieving a 95% success rate and a positional root mean square error (RMSE) of 0.129 m, outperforming existing adaptation methods with 86% success and an RMSE of 0.153 m. Further evaluations on quadrotors with varying configurations confirm the robustness of our framework across untrained dynamics. These results demonstrate the potential of our framework to enhance the adaptability and reliability of multirotors, enabling efficient fault management in dynamic and uncertain environments. Website is available at <http://00dhkim.me/paper/rl-ftc>

I. INTRODUCTION

Multirotors have emerged as effective tools in diverse field robotics applications, including logistics, surveillance, and agriculture. Stable and robust control of multirotors in challenging environments remains critical. In particular, multirotors generate lift through motor rotation, making them especially vulnerable to actuator faults such as loss-of-effectiveness (LoE) in certain motors. These faults immediately cause instability, making multirotors more vulnerable than fixed-wing aircraft. Therefore, actuator faults are critical in multirotor flight, requiring fault-tolerant control (FTC) strategies to ensure reliability.

Recently, FTC methods using reinforcement learning (RL) for multirotors have received attention due to their potential to learn complex and flexible control strategies. Existing studies have successfully demonstrated RL's capability to maintain system performance under various fault conditions, such as torque distribution [1] and underwater propeller faults [2], and fixed-wing UAV control [3]. Extensive research has particularly explored RL-based FTC for multirotors. For instance, a parallel combination of a model-free

RL-based FTC and original controllers was proposed as a countermeasure against UAV signal spoofing [4]. An RL-based policy that integrates outputs from both high-gain and low-gain PID controllers was developed [5]. A method for generating FTC signals by leveraging discrepancies between fault-affected sensed states and states from a predefined health model is introduced [6].

However, these studies face limitations when deployed in practical field scenarios. They often require prior system-specific health models or extensive retraining when encountering previously unseen dynamics, such as mid-mission changes in payload mass or variations in multirotor configuration. Such constraints significantly reduce the adaptability and flexibility required in realistic field robotics applications.

To address these challenges, we propose an RL-based hybrid FTC framework combined with an adaptation module inspired by the rapid motor adaptation (RMA) method [7]. Unlike previous approaches, our approach employs an adaptation module that adopts a control strategy suitable for the current drone configuration based on sensor data and command history. This method enables real-time, online adaptation without requiring additional training, thus facilitating adaptation to previously unseen dynamics, including mid-flight changes in mass or inertia. The adaptation module supports policies in adapting to diverse and uncertain environments. The RMA module developed by [7] has demonstrated successful real-time adaptation in quadruped robot control tasks. Extending from ground robots to aerial systems, recent studies by [8], [9] have achieved online adaptation for quadrotors, demonstrating robustness against environmental variations without additional training. Furthermore, [10] successfully adapted quadrotor control systems to external disturbances. While these studies emphasize environmental adaptation and resilience to minor oscillation, our work uniquely addresses significant actuator faults, such as substantial thrust losses reaching up to 36%.

Further enhancing our framework, we propose a novel transformer-based adaptation module to enhance domain inference capabilities beyond existing approaches. We validate our method in simulated hovering tasks using PyBullet [11], demonstrating robust and reliable position control despite substantial actuator faults. Our results emphasize the practical applicability of our proposed framework, advancing the robustness and flexibility of autonomous aerial systems.

II. METHOD

Our framework consists of a nominal controller, a policy, and an adaptation module. This section describes the problem

This work was supported by the Agency for Defense Development Grant funded by the Korean Government in 2025.

¹Dohyun Kim and Jungho Bae are with the Agency for Defense Development, Republic of Korea {00dh.kim, daewith}@gmail.com

²Jayden Dongwoo Lee and Hyochoong Bang are with the Korea Advanced Institute of Science and Technology, Republic of Korea {cin6474, hcbang}@kaist.ac.kr

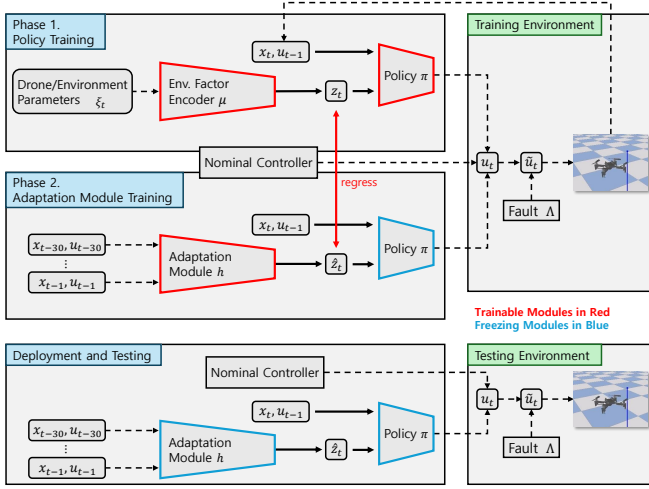


Fig. 1. The overview of 2-phase training and deployment. **Policy Training:** In phase 1, the environmental factor encoder μ and policy π are jointly trained. μ encodes privileged factors into a latent vector z_t . The policy then uses z_t together with state x_t and previous action u_{t-1} to generate the control command u_t^{nominal} . The command is added to the nominal controller's command u_t^{nominal} and applied to the dynamics model. **Adaptation Module Training:** The adaptation module h learns to infer $\hat{z}_t$ without access to privileged state ξ_t , through supervised learning to minimize the error between $\hat{z}_t$ and ground truth z_t . Training data is collected using the frozen policy π . **Deployment and Testing:** In deployment, both π and h are frozen. h infers $\hat{z}_t$ from trajectory, which π uses to produce control commands. This enables adaptability to unseen dynamics while achieving FTC.

formulation, the 2-phase training process, and adaptation module. For details, please see Fig. 1.

A. Problem Formulation

UAV Model Description: We use the physics-based gymbullet-drones environment [12] to simulate multirotor dynamics, including attitude and position changes. The thrust by each motor and torque induced around the multirotor's z-axis are expressed as follows:

$$F_i = k_F \cdot u_{t,i}^2, \quad T = \sum_{i=1}^4 (-1)^i k_T \cdot u_{t,i}^2, \quad (1)$$

where $u_t = [u_{t,1}, u_{t,2}, u_{t,3}, u_{t,4}]^T$ represents the revolutions per minute (RPM) of each motor, and k_F and k_T are the lift and torque coefficients, respectively.

Fault Representation: Actuators are subject to wear, propeller damage, and motor overheating. These faults are modeled as a loss-of-effectiveness, reducing the RPM of certain motors. The fault matrix Λ is defined as:

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \lambda_3, \lambda_4), \quad \tilde{u}_t = \Lambda u_t, \quad (2)$$

where λ_i represents the efficiency of the i -th motor and is nominally 1. The RPM with the fault is denoted as $\tilde{u}_t$. We consider faults affecting one motor with λ_i between 0.8 to 0.9, leading to up to 36% thrust loss. This reflects realistic fault scenarios [6]. Complete failure was not considered because this study focuses on partial damage situations.

B. Hybrid Controller

We propose a hybrid FTC controller for hovering under LoE actuator faults, combining a nominal controller and policy π . We also introduce an environmental factor encoder μ as a teacher model during adaptation module training.

Nominal Controller: The nominal controller uses a cascade Proportional-Integral-Differential (PID) structure for position and attitude control. The outer-loop PID computes z-axis thrust command and desired attitude from position errors, while the inner-loop PID generates torque commands from attitude errors. These commands are converted to RPM signals. These PID gains are optimized for accurate tracking under fault-free conditions and are used in both training and testing.

RL Policy: The policy π assists the nominal controller for stable hovering under actuator faults. At time t , it takes the state x_t , previous control input u_{t-1} , and latent vector z_t to output motor commands u_t . The state x_t includes position, attitude, linear velocity and angular velocity. The policy's control is capped at 20% of the nominal command, ensuring a supportive role in fault handling. z_t , extracted from the environmental encoder or adaptation module, allows for adaptive control. The control frequency is 30 Hz.

Latent Representation: The environmental factor encoder μ compresses privileged parameters ξ_t that represent the dynamic characteristics, allowing the policy to maintain consistent control performance across various domains. This approach aligns with previous research using domain adaptation methods [7], [8], [9] and effectively captures environmental variations. In addition, domain randomization is applied to facilitate generalization of both policy π and encoder μ . Specifically, ξ_t are randomly varied within $\pm 20\%$ to prevent the policy from overfitting to specific environments.

Reward Function: The reward function is designed to ensure stability despite faults. It minimizes position and attitude errors, penalizes rapid control fluctuations, and promotes progressive success. Let $\mathbf{p}$, $\mathbf{v}$, $\boldsymbol{\theta}$, and $\mathbf{p}_{\text{target}}$ denote the position, velocity, and attitude, and target position of multirotor, respectively. The reward at time t is defined as

$$R_t = -k_{\text{pos}} \|\mathbf{p} - \mathbf{p}_{\text{target}}\| - k_{\text{att}} \|\boldsymbol{\theta}\| - k_{\text{vel}} \|\mathbf{v}\|^2 - k_{\text{rate}} \|\dot{u}_t\|^2 - k_{\text{smooth}} \|\ddot{u}_t\|^2 + 0.001 + R_{\text{succ}} \quad (3)$$

$$R_{\text{succ}} = \begin{cases} 1.0, & \text{if } \|\mathbf{p} - \mathbf{p}_{\text{target}}\| < \epsilon_{\text{goal}} \text{ for 1 sec.} \\ 0.1, & \text{if } \|\mathbf{p} - \mathbf{p}_{\text{target}}\| < \epsilon_{\text{goal}} \\ 0.05, & \text{if } \|\mathbf{p} - \mathbf{p}_{\text{target}}\| < \epsilon_{\text{near}} \\ 0, & \text{otherwise,} \end{cases} \quad (4)$$

where the scaling factors are set to 0.5, 0.1, 0.001, 0.5, and 0.1, respectively, and the thresholds are set to $\epsilon_{\text{goal}} = 0.173$ and $\epsilon_{\text{near}} = 0.520$.

C. Transformer-based Adaptation Module

Measuring quadrotor and environmental parameters ξ_t in the real world is challenging. To address this, we replace

TABLE I
RANGES OF THE PHYSICAL PARAMETERS AND CONFIGURATION.

Parameter	CF	Test	
		UAV1	UAV2
Mass (g)	27	40.5	18.9
Arm Length (m)	0.0397	0.0594	0.0397
Thrust Coef. ($\text{N}/(\text{rad/s})^2, \times 10^{-10}$)	3.16	4.74	2.21
Torque Coef. ($\text{N} \cdot \text{m}/(\text{rad/s})^2, \times 10^{-12}$)	7.94	1.19	5.56
Thrust-to-Weight Ratio	2.25	2	2.8
Inertia I_{xx} ($\text{g} \cdot \text{m}^2, \times 10^{-2}$)	2.3951	4.73	0.98
Inertia I_{yy} ($\text{g} \cdot \text{m}^2, \times 10^{-2}$)	2.3951	4.73	0.98
Inertia I_{zz} ($\text{g} \cdot \text{m}^2, \times 10^{-2}$)	3.2347	7.32	1.52
Initial Position(X, Y) (m)	$[-0.2, 0.2]$	$[-0.6, 0.6]$	
Initial Position(Z) (m)	$[0.8, 1.2]$	$[0.7, 1.3]$	
Fault Range	$[0.1, 0.2]$	$[0.1, 0.2]$	

TABLE II
PERFORMANCE OF OUR METHOD AND BASELINES FOR EACH
QUADROTOR UNDER 30% LOE OF F_2 .

Model	Method	Success	RMSE (m)	Max Error (m)
CF	RL-ours	95%	0.129 ± 0.042	0.545 ± 0.135
	RL-RMA [7]	86%	0.153 ± 0.092	0.565 ± 0.145
	PID	53%	1.220 ± 1.605	2.230 ± 2.178
	No Fault (oracle)	100%	0.063 ± 0.015	0.486 ± 0.160
UAV1	RL-ours	91%	0.165 ± 0.048	0.589 ± 0.132
	RL-RMA [7]	85%	0.182 ± 0.092	0.615 ± 0.149
	PID	55%	1.340 ± 1.629	2.460 ± 2.231
	No Fault (oracle)	100%	0.074 ± 0.013	0.487 ± 0.159
UAV2	RL-ours	96%	0.109 ± 0.042	0.534 ± 0.150
	RL-RMA [7]	87%	0.136 ± 0.093	0.554 ± 0.156
	PID	45%	1.286 ± 1.802	2.294 ± 2.587
	No Fault (oracle)	100%	0.057 ± 0.016	0.486 ± 0.161

the encoder μ with an adaptation module. To the best of our knowledge, this paper is the first application of RMA-style adaptation module for fault-tolerant control, enabling robust adaptive control without privileged information. Our adaptation module h infers a latent vector $\hat{z}_t$ from 1 second state-action trajectories. The module is trained using a teacher-student paradigm. The frozen environmental encoder μ acts as the teacher without training, and h learns to mimic its output by minimizing the mean squared error $\|\hat{z}_t - z_t\|^2$. Training data is collected using the frozen policy learned in Phase 1.

Unlike RMA [7], we introduce a transformer architecture [13] to improve inference performance. Transformers leverage self-attention mechanisms to capture temporal correlations throughout the entire input sequence. This approach effectively filters noise and vibration, and discriminates key physical characteristics. This facilitates the stability of latent representations, ensuring robust inference performance even in unseen dynamics during training.

III. EXPERIMENTS

A. Experiment Setup

We evaluated our framework in gym-pybullet-drones [12], a high-fidelity simulator. The simulation used the Bitcraze Crazyflie 2.x (referred to as **CF**) as the base quadrotor, and evaluated adaptability with two variants: a larger and heavier

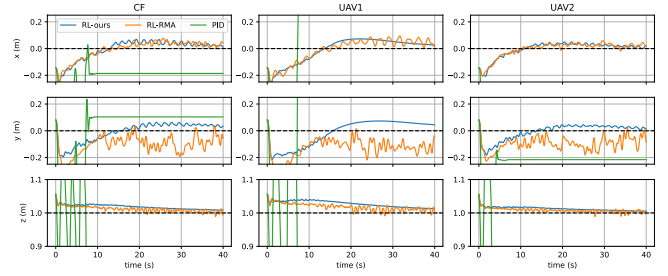


Fig. 2. State comparison between our method and the baselines for each quadrotor under 30% LoE of F_2 . The black dashed lines are the target state.

UAV1 and a lighter **UAV2**. Their physical parameters are in Table I.

In Phase 1, the policy π and environmental encoder μ were jointly trained with proximal policy optimization [14] algorithm. In Phase 2, the adaptation module h was trained using the ADAM [15] optimizer. The environmental encoder used a 2-layer MLP with a 64-dimensional hidden layer. The policy followed the Stable Baselines3 [16] settings. To compare adaptation module structures, both CNN-based and transformer-based modules were designed. The CNN model had a 3-layer 1-D CNN architecture with input/output channel numbers, kernel sizes, and strides set to $[32, 32, 4, 2]$, $[32, 32, 3, 1]$, and $[32, 32, 3, 1]$, respectively. The transformer model was configured with a 16-dimensional input, 128 model dimension, sequence length 64, 2 attention heads, 2 encoder layers, a 128 feed-forward dimension, and an 8-dimensional latent output.

B. Experiment Results

We validate our framework in a simulation and compare it with two other methods. **RL-ours** utilizes our hybrid controller and transformer-based adaptation module. **RL-RMA** [7] shares the same controller but employs a 1-D CNN-based RMA-style adaptation module for benchmarking. **PID** is a standard Proportional-Integral-Derivative controller without fault handling. **No Fault (oracle)** evaluates performance without faults, representing the theoretical upper bound for RL-ours, which aims to match this level under optimal fault-tolerant control.

State Comparison: We compare control performance under a 30% thrust loss in motor 2 across three quadrotor models: CF, UAV1, and UAV2. RL-ours demonstrates successful convergence within the margin, while RL-RMA shows significant oscillations and exceeds it. PID becomes unstable and crashed after about 7 seconds. Fig. 2 shows state trajectories.

Quantitative Testing: We conduct quantitative tests on 100 random initial states, with positions and fault magnitudes sampled from Table I. Metrics include control success probability, root mean square error (RMSE) of position, and maximum position error. Success is defined as maintaining a positional error within 0.173m for at least one second. As shown in Table II, our method outperforms others across all

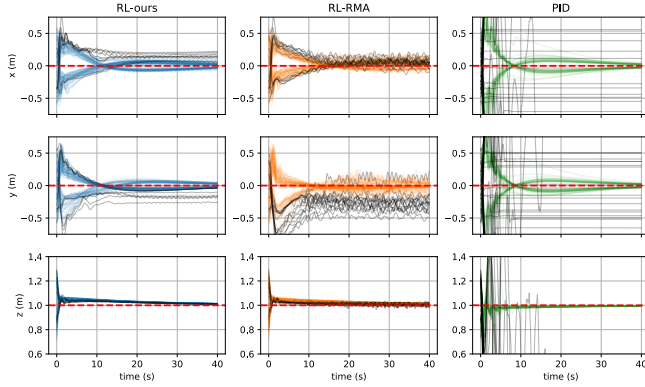


Fig. 3. State comparison between our method and the baselines for the UAV1 model with 100 fault cases. Colored solid lines are successful cases, black solid lines are failure cases, and red dotted lines are the target state.

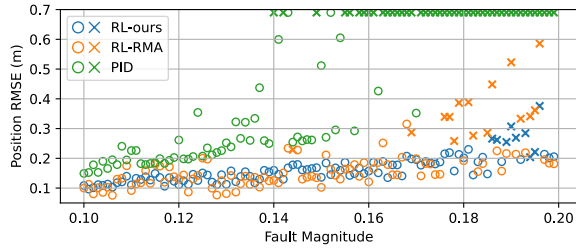


Fig. 4. Position RMSE and success/failure results between our method and the baselines for the UAV1 model with 100 fault cases. The cases where the RMSE exceeds 0.7m are noted on the boundary.

metrics, showing strong fault tolerance. It achieves precise control with high success, even on untrained UAVs, demonstrating its adaptability.

Robustness Testing: We evaluate robustness by testing across the fault range using UAV1, an unseen model, with 100 test cases. Fig. 3 shows state trajectories. RL-ours stabilizes near targets, while the others often deviates significantly or oscillates. Fig. 4 demonstrates position RMSE and success status per case. RL-ours maintains lower RMSE and higher success, even with significant LoE faults above 0.18. The others struggle as fault magnitude increases. This indicates our method’s consistent fault tolerance, even in substantial faults and unseen dynamics.

Latent Inference: Finally, we compare the latent inference performance of the proposed transformer-based adaptation module with the RMA-style 1-D CNN. RMSE $\|\hat{z}_t - z_t\|$ is measured between the adaptation module and the environmental factor encoder. RL-ours achieves an RMSE of 0.1113 ± 0.0036 , far lower than 5.5798 ± 0.0009 of RL-RMA, demonstrating improved inference accuracy. Notably, our method maintains precise latent inference, which confirms that our module effectively replaces the environmental encoder without privileged parameters.

IV. CONCLUSIONS

We presented a reinforcement learning-based fault-tolerant control under uncertain and varying field conditions. Inspired

by recent advancements in RMA, our proposed framework enables robust real-time inference without the need for additional privileged information or retraining. Experiments demonstrate that our method outperforms PID and CNN-based approaches across various actuator faults and unseen dynamic scenarios, including loss-of-effectiveness faults and variations in quadrotor physical parameters. Our method maintains superior stability even beyond the training distribution. These advantages make our framework reliable for practical field applications, even when the payload changes unexpectedly, thereby reducing mission complexity.

REFERENCES

- [1] H. Deng, Y. Zhao, A.-T. Nguyen, and C. Huang, “Fault-tolerant predictive control with deep-reinforcement-learning-based torque distribution for four in-wheel motor drive electric vehicles,” *IEEE/ASME Transactions on Mechatronics*, vol. 28, no. 2, pp. 668–680, 2023.
- [2] J. Qin, M. Zhong, W. Gai, and Z. Ding, “Fault-tolerant trajectory tracking control based on ddpg algorithm for underwater vehicle with propeller faults,” *International Journal of Control, Automation and Systems*, vol. 22, no. 4, pp. 1418–1429, 2024.
- [3] F. Giral, I. Gómez, R. Vinuesa, and S. Le-Clainche, “Transformer-based fault-tolerant control for fixed-wing uavs using knowledge distillation and in-context adaptation,” *arXiv preprint arXiv:2411.02975*, 2024.
- [4] F. Fei, Z. Tu, D. Xu, and X. Deng, “Learn-to-recover: Retrofitting uavs with reinforcement learning-assisted flight control under cyber-physical attacks,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 7358–7364.
- [5] Y. Sohège, M. Quinones-Grueiro, and G. Provan, “A novel hybrid approach for fault-tolerant control of uavs based on robust reinforcement learning,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 10719–10725.
- [6] X. Liu, Z. Yuan, Z. Gao, and W. Zhang, “Reinforcement learning-based fault-tolerant control for quadrotor uavs under actuator fault,” *IEEE Transactions on Industrial Informatics*, 2024.
- [7] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “Rma: Rapid motor adaptation for legged robots,” *arXiv preprint arXiv:2107.04034*, 2021.
- [8] D. Zhang, A. Loquercio, X. Wu, A. Kumar, J. Malik, and M. W. Mueller, “Learning a single near-hover position controller for vastly different quadcopters,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1263–1269.
- [9] D. Zhang, A. Loquercio, J. Tang, T.-H. Wang, J. Malik, and M. W. Mueller, “A learning-based quadcopter controller with extreme adaptation,” *arXiv preprint arXiv:2409.12949*, 2024.
- [10] H.-L. Hsu, H. Meng, S. Luo, J. Dong, V. Tarokh, and M. Pajic, “Reforma: Robust reinforcement learning via adaptive adversary for drones flying under disturbances,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 5169–5175.
- [11] E. Coumans and Y. Bai, “Pybullet, a python module for physics simulation for games, robotics and machine learning,” <http://pybullet.org>, 2016–2021.
- [12] J. Panerati, H. Zheng, S. Zhou, J. Xu, A. Prorok, and A. P. Schoellig, “Learning to fly—a gym environment with pybullet physics for reinforcement learning of multi-agent quadcopter control,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 7512–7519.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [16] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of machine learning research*, vol. 22, no. 268, pp. 1–8, 2021.